

PINN-MLP vs PINN-KAN: Ecuación de Burgers Estacionaria 1D

Análisis completo del código — Matemáticas, gradientes y corrección exhaustiva

1. El Problema Físico: Ecuación de Burgers Estacionaria

1.1 La Ecuación

La ecuación de Burgers estacionaria 1D es:

$$u(x) \cdot u_x(x) = \nu \cdot u_{xx}(x), \quad x \in [-1, 1]$$
$$u(-1) = 1, \quad u(1) = -1$$

donde ν (nu) es la viscosidad cinemática, fijada a 0.3 en el código.

1.2 ¿Por qué es difícil esta ecuación?

A diferencia de la ecuación de Poisson (que es lineal), la ecuación de Burgers es no lineal: el término $u \cdot u_x$ acopla la solución con su propia derivada. Esta no linealidad tiene consecuencias profundas:

Consecuencias de la no linealidad

1. El residuo $R = u \cdot u_x - \nu \cdot u_{xx}$ es cuadrático en u . Sus gradientes son más complejos porque hay regla del producto.
2. Para ν pequeño, la solución desarrolla una onda de choque (capa límite) muy pronunciada en $x=0$. El código advierte esto: con $\nu=0.3$, la transición es suave.
3. Existen múltiples soluciones locales. El descenso de gradiente puede quedar atrapado.

1.3 Solución Exacta Aproximada

```
u_exact = -np.tanh(x / (2 * nu))
```

Para verificar: si $u = -\tanh(x/(2\nu))$, entonces:

$$u_x = -\operatorname{sech}^2(x/2\nu) \cdot (1/2\nu)$$
$$u_{xx} = \operatorname{sech}^2(x/2\nu) \cdot \tanh(x/2\nu) \cdot (1/\nu^2)$$

Sustituyendo en la EDP:

$$u \cdot u_x = \tanh \cdot \operatorname{sech}^2 / 2\nu$$
$$\nu \cdot u_{xx} = \operatorname{sech}^2 \cdot \tanh / \nu$$

Estos no son iguales en general — la solución $-\tanh(x/2\nu)$ satisface la EDP de Burgers exactamente solo si se verifican ciertas condiciones. Para $\nu=0.3$ y las CC $u(-1)=1$, $u(1)=-1$, es una aproximación razonable pero no exacta. El código lo llama «exacta aproximada».

⚠ La solución exacta no es del todo exacta

$u = -\tanh(x/2\nu)$ es la solución analítica de la ecuación de Burgers viscosa $u \cdot u_x = \nu \cdot u_{xx}$ para las CC $u(\pm\infty) = \mp 1$ (dominio infinito). En el dominio finito $[-1,1]$, es solo una muy buena aproximación, especialmente para ν pequeño. El código lo reconoce con el comentario 'aproximada'.

1.4 Configuración Numérica

```
N_points = 100
x = np.linspace(-1, 1, N_points).reshape(-1, 1)  # (100,1)
nu = 0.3
u_exact = -np.tanh(x / (2 * nu))
epochs = 1000
```

x = linspace(-1,1,100): 100 colocation points uniformes en $[-1,1]$.

nu = 0.3: Viscosidad moderada. Con $\nu < 0.05$ aparece una capa límite casi vertical, muy difícil para GD sin Adam u optimizadores adaptativos.

2. PINN-MLP para Burgers — Análisis Completo

2.1 Inicialización

```
H = 20
np.random.seed(42)
W = np.random.randn(1, H) * 0.1  # (1, 20)
b = np.zeros((1, H))             # (1, 20)
V = np.random.randn(H, 1) * 0.1  # (20, 1)
lr_mlp = 0.005
```

Igual que en el código de Poisson. Una capa oculta con 20 neuronas tanh. Los pesos pequeños ($\times 0.1$) evitan saturación inicial de tanh.

2.2 Forward Pass: u, u_x, u_xx

La predicción y sus derivadas necesitan calcularse hasta segundo orden, y el backprop necesita terceras derivadas:

```
z = np.dot(x, W) + b  # (100,20) preactivación
a = np.tanh(z)         # (100,20) activación
u_pred = np.dot(a, V)  # (100,1) predicción
```

Derivadas de tanh:

$$\begin{aligned}\tanh'(z) &= 1 - \tanh^2(z) \equiv \operatorname{sech}^2(z) \\ \tanh''(z) &= -2 \cdot \tanh(z) \cdot \operatorname{sech}^2(z) \\ \tanh'''(z) &= -2 \cdot \operatorname{sech}^2(z) \cdot (1 - 3 \cdot \tanh^2(z))\end{aligned}$$

La tercera derivada es necesaria porque al derivar u_{xx} respecto a b o W , aparece $\tanh'''$ en la regla de la cadena.

```

a_prime      = 1 - a**2                # (100,20) tanh'
a_double_prime = -2 * a * a_prime      # (100,20) tanh''
a_triple_prime = -2 * a_prime * (1 - 3 * a**2) # (100,20) tanh'''

```

Derivación de a_triple_prime :

Partiendo de $g(z) = \tanh'(z) = -2 \cdot \tanh(z) \cdot \text{sech}^2(z)$, derivamos respecto a z :

$$\begin{aligned}
 g'(z) &= -2 \cdot [\text{sech}^2(z) \cdot \text{sech}^2(z) + \tanh(z) \cdot (-2 \cdot \text{sech}^2(z) \cdot \tanh(z))] \\
 &= -2 \cdot \text{sech}^2(z) \cdot [\text{sech}^2(z) - 2 \cdot \tanh^2(z)] \\
 &= -2 \cdot \text{sech}^2(z) \cdot [(1-a^2) - 2a^2] \\
 &= -2 \cdot (1-a^2) \cdot (1-3a^2) \\
 &= -2 \cdot a_prime \cdot (1-3a^2)
 \end{aligned}$$

✓ Esto coincide exactamente con el código: $a_triple_prime = -2 \cdot a_prime \cdot (1-3 \cdot a^2)$

Derivadas de u :

Dado que $u = \sum_j V_j \cdot \tanh(W_j \cdot x + b_j)$:

$$\begin{aligned}
 u_x &= \sum_j V_j \cdot \tanh'(z_j) \cdot W_j = \sum_j V_j \cdot a'_j \cdot W_j \\
 u_{xx} &= \sum_j V_j \cdot \tanh''(z_j) \cdot W_j^2 = \sum_j V_j \cdot a''_j \cdot W_j^2
 \end{aligned}$$

```

u_x = np.dot(a_prime * W, V)          # (100,20)*(1,20)→elem→(100,20) · (20,1) = (100,1)
u_xx = np.dot(a_double_prime * (W**2), V) # (100,1)

```

La operación $a_prime * W$ es broadcasting: $(100,20) \times (1,20) \rightarrow (100,20)$, donde elemento $(i,j) = a'_{ij} \cdot W_j$. Luego $\text{dot}(\cdot, V)$ suma sobre j : $\sum_j a'_{ij} \cdot W_j \cdot V_j = u_x(x_i)$. ✓

2.3 Función de Pérdida

```

R = u_pred * u_x - nu * u_xx          # (100,1) - residuo de Burgers
L_pde = np.mean(R**2)                 # MSE del residuo
R_bc0 = u_pred[0,0] - 1.0             # Residuo en x=-1: u(-1)-1
R_bc1 = u_pred[-1,0] - (-1.0)        # Residuo en x=+1: u(1)+1
L_bc = R_bc0**2 + R_bc1**2

```

Residuo $R = u \cdot u_x - \nu \cdot u_{xx}$: Es la cantidad que debe ser cero si u satisface la EDP. Al ser u no lineal, R es cuadrático en los parámetros, lo que complica la optimización.

$R_bc0 = u_pred[0,0] - 1.0$: Diferencia entre la predicción en $x=-1$ y el valor objetivo 1. A diferencia del código de Poisson donde $L_bc = u(0)^2 + u(\pi)^2$, aquí los targets no son cero, sino +1 y -1.

Diferencia clave con Poisson

```

Poisson:  L_BC = u(0)2 + u(π)2          - target = 0, residuo = u(xbc)
Burgers:  L_BC = (u(-1)-1)2 + (u(1)+1)2 - target ≠ 0, residuo = u(xbc) - target

```

Esto afecta directamente los gradientes de las CC, que ahora incluyen el residuo R_bc en lugar de u_pred .

2.4 Gradientes MLP — PDE (Regla del Producto)

La pérdida PDE es $L_{PDE} = (1/N) \cdot \sum_i R_i^2$ con $R = u \cdot u_x - v \cdot u_{xx}$. Derivando respecto a parámetro θ :

$$\begin{aligned} \partial L_{PDE} / \partial \theta &= (2/N) \cdot \sum_i R_i \cdot \partial R_i / \partial \theta \\ \partial R / \partial \theta &= u_x \cdot (\partial u / \partial \theta) + u \cdot (\partial u_x / \partial \theta) - v \cdot (\partial u_{xx} / \partial \theta) \end{aligned}$$

Esta es la regla del producto para la no linealidad $u \cdot u_x$. Cada derivada requiere tres términos.

Gradiente respecto a V

V aparece linealmente en u, u_x y u_{xx} (la no linealidad está en $u \cdot u_x$, no en la dependencia de V):

$$\begin{aligned} \partial u / \partial V_j &= a_j \rightarrow d_u_{dV} = a \quad (100, 20) \\ \partial u_x / \partial V_j &= a'_j \cdot W_j \rightarrow d_{ux}_{dV} = a_{\text{prime}} * W \quad (100, 20) \\ \partial u_{xx} / \partial V_j &= a''_j \cdot W_j^2 \rightarrow d_{uxx}_{dV} = a_{\text{dbl}} * (W^2) \quad (100, 20) \end{aligned}$$

```
d_u_dV = a # (100,20)
d_ux_dV = a_prime * W # (100,20) broadcasting (100,20)*(1,20)
d_uxx_dV = a_double_prime * (W**2) # (100,20)
d_R_dV = u_x*d_u_dV + u_pred*d_ux_dV - nu*d_uxx_dV # (100,20)
grad_V = np.mean(2*R*d_R_dV, axis=0, keepdims=True).T # (20,1)
```

La multiplicación $2 * R * d_R_{dV}$: R tiene forma (100,1) y d_R_{dV} tiene (100,20). El broadcasting replica R en las 20 columnas $\rightarrow$ (100,20). La media sobre axis=0 da (1,20), y .T da (20,1) = forma de V. ✓

Gradiente respecto a b

b es el sesgo: $z_j = W_j \cdot x + b_j$, por tanto $\partial z_j / \partial b_j = 1$

$$\begin{aligned} \partial u / \partial b_j &= V_j \cdot \tanh'(z_j) \cdot 1 = V_j \cdot a'_j \\ \partial u_x / \partial b_j &= V_j \cdot \tanh''(z_j) \cdot W_j = V_j \cdot a''_j \cdot W_j \\ \partial u_{xx} / \partial b_j &= V_j \cdot \tanh'''(z_j) \cdot W_j^2 = V_j \cdot a'''_j \cdot W_j^2 \end{aligned}$$

```
d_u_db = a_prime * V.T # (100,20)*(1,20)=(100,20)
d_ux_db = a_double_prime * W * V.T # (100,20)
d_uxx_db = a_triple_prime * (W**2) * V.T # (100,20)
d_R_db = u_x*d_u_db + u_pred*d_ux_db - nu*d_uxx_db
grad_b = np.mean(2*R*d_R_db, axis=0, keepdims=True) # (1,20)
```

Aquí aparece `a_triple_prime` porque u_{xx} contiene a'' , y al derivar respecto a b necesitamos la derivada de a'' , que es a''' . Esta es la razón por la que se definió la tercera derivada de tanh.

Gradiente respecto a W

W aparece en z tanto directamente ($W_j \cdot x$) como en todas las derivadas:

$$\begin{aligned} \partial u / \partial W_j &= V_j \cdot a'_j \cdot x \\ \partial u_x / \partial W_j &= V_j \cdot [a'_j + W_j \cdot x \cdot a''_j] \quad (\text{regla del producto}) \\ \partial u_{xx} / \partial W_j &= V_j \cdot [2W_j \cdot a''_j + W_j^2 \cdot x \cdot a'''_j] \quad (\text{regla del producto}) \end{aligned}$$

Derivación de $\partial u_x / \partial W_j$:

Como $u_x = \sum_k V_k \cdot a'_k \cdot W_k$, solo el término $k=j$ depende de W_j :

$$\begin{aligned} \partial / \partial W_j [V_j \cdot a'_j \cdot W_j] &= V_j \cdot [a''_j \cdot (\partial z_j / \partial W_j) \cdot W_j + a'_j \cdot 1] \\ &= V_j \cdot [a''_j \cdot x \cdot W_j + a'_j] \\ &= V_j \cdot [a'_j + W_j \cdot x \cdot a''_j] \end{aligned}$$

Derivación de $\partial u_{xx}/\partial W_j$:

Como $u_{xx} = \sum_k V_k \cdot a''_k \cdot W_k^2$:

$$\begin{aligned} \frac{\partial}{\partial W_j} [V_j \cdot a''_j \cdot W_j^2] &= V_j \cdot [a'''_j \cdot x \cdot W_j^2 + a''_j \cdot 2W_j] \\ &= V_j \cdot [2W_j \cdot a''_j + W_j^2 \cdot x \cdot a'''_j] \end{aligned}$$

```
d_u_dW = x * a_prime * V.T # (100,20)
d_ux_dW = a_prime*V.T + x*a_double_prime*W*V.T # (100,20)
d_uxx_dW = 2*a_double_prime*W*V.T + x*a_triple_prime*(W**2)*V.T # (100,20)
d_R_dW = u_x*d_u_dW + u_pred*d_ux_dW - nu*d_uxx_dW
grad_W = np.mean(2*R*d_R_dW, axis=0, keepdims=True) # (1,20)
```

2.5 Gradientes de las Condiciones de Contorno — MLP

La pérdida de CC es $L_{CC} = R_{bc0}^2 + R_{bc1}^2$ con $R_{bc0} = u(-1) - 1$ y $R_{bc1} = u(1) - (-1)$.

Derivando:

$$\partial L_{CC}/\partial \theta = 2 \cdot R_{bc0} \cdot (\partial u(-1)/\partial \theta) + 2 \cdot R_{bc1} \cdot (\partial u(1)/\partial \theta)$$

A diferencia de Poisson donde los residuos CC eran $u(x_{bc})$ directamente, aquí el factor delante de $\partial u/\partial \theta$ es $R_{bc} = u(x_{bc}) - \text{target}$. Matemáticamente la regla de la cadena funciona igual, solo cambia el «prefactor».

```
grad_V += 2*R_bc0*d_u_dV[0:1,:].T + 2*R_bc1*d_u_dV[-1:,:].T
grad_b += 2*R_bc0*d_u_db[0:1,:] + 2*R_bc1*d_u_db[-1:,:]
grad_W += 2*R_bc0*d_u_dW[0:1,:] + 2*R_bc1*d_u_dW[-1:,:]
```

d_u_dV[0:1,:].T: Extrae la fila 0 ($x=-1$) de d_u_dV , que es $a[0:1,:]$ con forma (1,20), y la transpone a (20,1) = forma de V . ✓

d_u_db[0:1,:]: Extrae la fila 0 de d_u_db ($= a_{\text{prime}}[0]*V.T$), forma (1,20) = forma de b . ✓

d_u_dW[0:1,:]: Extrae la fila 0 de d_u_dW ($= x[0]*a_{\text{prime}}[0]*V.T$). Como $x[0]=-1$, añade el signo negativo. ✓

3. PINN-KAN para Burgers — Análisis Completo

3.1 Inicialización KAN

```
C = np.zeros((H, 1)) # (20,1)
mu = np.linspace(-1, 1, H).reshape(1, H) # (1,20) centros en [-1,1]
sigma = np.ones((1, H)) * (2.0/H) * 1.5 # (1,20) anchuras

lr_C = lr_mu = lr_sigma = 0.005
```

mu = linspace(-1,1,20): Centros distribuidos uniformemente por el dominio $[-1,1]$ (antes era $[0,\pi]$).

sigma = (2/H)*1.5: La longitud del dominio ahora es 2 (de -1 a 1). La separación entre centros es $2/H$, y $\sigma = 1.5$ veces esa separación para garantizar solapamiento.

3.2 Forward Pass KAN: u , u_x , u_{xx}

La red KAN predice: $u(x) = \sum_j C_j \cdot \varphi_j(x)$ donde $\varphi_j(x) = \exp(-0.5 \cdot z_j^2)$ y $z_j = (x - \mu_j) / \sigma_j$

```
z      = (x - mu) / sigma          # (100,1)-(1,20) / (1,20) = (100,20)
phi    = np.exp(-0.5 * z**2)      # (100,20)
u_pred_kan = np.dot(phi, C)       # (100,1)

z2 = z**2
phi_x  = phi * (-z / sigma)       # (100,20) - primera derivada de φ
phi_xx = phi * (z2 - 1) / (sigma**2) # (100,20) - segunda derivada de φ

u_x_kan = np.dot(phi_x, C)        # (100,1)
u_xx_kan = np.dot(phi_xx, C)      # (100,1)
```

phi_x = φ·(-z/σ): Derivada de $\exp(-z^2/2)$ respecto a x:

$$\partial \varphi_j / \partial x = \exp(-z^2/2) \cdot (-z) \cdot (\partial z / \partial x) = \varphi_j \cdot (-z_j) \cdot (1/\sigma_j) = -\varphi_j \cdot z_j / \sigma_j$$

phi_xx = φ·(z²-1)/σ²: Segunda derivada (ya derivada en el documento de Poisson):

$$\partial^2 \varphi_j / \partial x^2 = \varphi_j \cdot (z_j^2 - 1) / \sigma_j^2$$

La primera derivada `phi_x` es nueva respecto al código de Poisson, necesaria porque Burgers tiene el término $u \cdot u_x$.

3.3 Función de Pérdida KAN

```
R_kan    = u_pred_kan * u_x_kan - nu * u_xx_kan # (100,1)
L_pde_kan = np.mean(R_kan**2)
R_bc0_kan = u_pred_kan[0,0] - 1.0
R_bc1_kan = u_pred_kan[-1,0] - (-1.0)
L_bc_kan  = R_bc0_kan**2 + R_bc1_kan**2
```

La misma estructura que la MLP: residuo de Burgers más residuos de las CC con targets ± 1 .

3.4 Derivadas de φ respecto a μ y σ (hasta orden 2 en x)

Para calcular los gradientes de la PDE respecto a μ y σ , necesitamos las derivadas cruzadas de φ respecto a x y a los parámetros. Se definen seis funciones auxiliares:

Derivadas respecto a μ

Nota: $\partial z_j / \partial \mu_j = -1/\sigma_j$

φ_μ = ∂φ/∂μ:

$$\partial \varphi / \partial \mu_j = \varphi \cdot (-z_j) \cdot (\partial z_j / \partial \mu_j) = \varphi \cdot (-z_j) \cdot (-1/\sigma_j) = \varphi \cdot z_j / \sigma_j$$

φ_xμ = ∂²φ/∂x∂μ = ∂(φ_x)/∂μ:

Partiendo de $\varphi_x = -\varphi \cdot z_j / \sigma_j$:

$$\begin{aligned} \partial / \partial \mu_j [-\varphi \cdot z_j / \sigma_j] &= -(\varphi_\mu \cdot z_j / \sigma_j + \varphi \cdot (\partial z_j / \partial \mu_j) / \sigma_j) \\ &= -(\varphi \cdot z_j / \sigma_j \cdot z_j / \sigma_j + \varphi \cdot (-1/\sigma_j) / \sigma_j) \\ &= -(\varphi \cdot z_j^2 / \sigma_j^2 - \varphi / \sigma_j^2) \\ &= \varphi \cdot (1 - z_j^2) / \sigma_j^2 \end{aligned}$$

$\varphi_{xx\mu} = \partial^2 \varphi_{xx} / \partial \mu = \partial / \partial \mu [\varphi \cdot (z^2 - 1) / \sigma^2]$:

$$\begin{aligned} & \partial / \partial \mu_j [\varphi \cdot (z_j^2 - 1) / \sigma_j^2] \\ &= (\varphi \cdot z_j / \sigma_j) \cdot (z_j^2 - 1) / \sigma_j^2 + \varphi \cdot (2z_j \cdot (-1 / \sigma_j)) / \sigma_j^2 \\ &= \varphi / \sigma_j^3 \cdot [z_j \cdot (z_j^2 - 1) - 2z_j] \\ &= \varphi \cdot z_j \cdot (z_j^2 - 3) / \sigma_j^3 \\ &= \varphi \cdot (z_j^3 - 3z_j) / \sigma_j^3 \end{aligned}$$

```
phi_mu    = phi * (z / sigma)           # ∂φ/∂μ
phi_xmu   = phi * (1 - z2) / (sigma**2) # ∂(φ_x)/∂μ
phi_xxmu  = phi * (3*z - z**3) / (sigma**3) # ∂(φ_xx)/∂μ
```

Nótese que $\text{phi_xxmu} = \text{phi} \cdot (3z - z^3) / \sigma^3 = -\text{phi} \cdot (z^3 - 3z) / \sigma^3$. Compara con el código de Poisson que tenía $\text{d_uxx_dmu} = C \cdot \text{phi} \cdot (z^3 - 3z) / \sigma^3$. Aquí se absorbe el signo porque se calcula la derivada de φ_{xx} respecto a μ antes de multiplicar por C , y el signo se gestiona al componer $\text{d_uxx_dmu} = \text{phi_xxmu} \cdot C \cdot T$.

Derivadas respecto a σ

Nota: $\partial z_j / \partial \sigma_j = -(x - \mu_j) / \sigma_j^2 = -z_j / \sigma_j$

$\varphi_\sigma = \partial \varphi / \partial \sigma$:

$$\partial \varphi / \partial \sigma_j = \varphi \cdot (-z_j) \cdot (-z_j / \sigma_j) = \varphi \cdot z_j^2 / \sigma_j$$

$\varphi_{x\sigma} = \partial(\varphi_x) / \partial \sigma$:

Partiendo de $\varphi_x = -\varphi \cdot z_j / \sigma_j$:

$$\begin{aligned} & \partial / \partial \sigma_j [-\varphi \cdot z_j / \sigma_j] \\ &= -(\varphi_\sigma \cdot z_j / \sigma_j + \varphi \cdot (\partial z_j / \partial \sigma_j) / \sigma_j + \varphi \cdot z_j \cdot (-1 / \sigma_j^2)) \\ &= -(\varphi \cdot z_j^2 / \sigma_j \cdot z_j / \sigma_j + \varphi \cdot (-z_j / \sigma_j) / \sigma_j + \varphi \cdot z_j \cdot (-1 / \sigma_j^2)) \\ &= -(\varphi / \sigma_j^2 \cdot [z_j^3 - z_j - z_j]) \\ &= \varphi \cdot (2z_j - z_j^3) / \sigma_j^2 \end{aligned}$$

$\varphi_{xx\sigma} = \partial(\varphi_{xx}) / \partial \sigma$:

Partiendo de $\varphi_{xx} = \varphi \cdot (z_j^2 - 1) / \sigma_j^2$:

$$\begin{aligned} & \partial / \partial \sigma_j [\varphi \cdot (z_j^2 - 1) / \sigma_j^2] \\ &= \varphi_\sigma \cdot (z_j^2 - 1) / \sigma_j^2 + \varphi \cdot (2z_j \cdot (-z_j / \sigma_j)) / \sigma_j^2 + \varphi \cdot (z_j^2 - 1) \cdot (-2 / \sigma_j^3) \\ &= \varphi \cdot z_j^2 / \sigma_j \cdot (z_j^2 - 1) / \sigma_j^2 - 2\varphi \cdot z_j^2 / \sigma_j^3 - 2\varphi \cdot (z_j^2 - 1) / \sigma_j^3 \\ &= \varphi / \sigma_j^3 \cdot [z_j^2 (z_j^2 - 1) - 2z_j^2 - 2(z_j^2 - 1)] \\ &= \varphi / \sigma_j^3 \cdot [z_j^4 - z_j^2 - 2z_j^2 - 2z_j^2 + 2] \\ &= \varphi \cdot (z_j^4 - 5z_j^2 + 2) / \sigma_j^3 \end{aligned}$$

```
phi_sigma    = phi * (z2 / sigma)           # ∂φ/∂σ
phi_xsigma   = phi * (2*z - z**3) / (sigma**2) # ∂(φ_x)/∂σ
phi_xxsigma  = phi * (z**4 - 5*z2 + 2) / (sigma**3) # ∂(φ_xx)/∂σ
```

3.5 Gradientes KAN — PDE (Regla del Producto)

El residuo es $R = u \cdot u_x - v \cdot u_{xx}$ donde $u = \sum_j C_j \cdot \varphi_j$. Para cada parámetro θ :

$$\partial R / \partial \theta = u_x \cdot (\partial u / \partial \theta) + u \cdot (\partial u_x / \partial \theta) - v \cdot (\partial u_{xx} / \partial \theta)$$

Gradiente respecto a C

```

$$\begin{aligned}\partial u / \partial C_j &= \varphi_j \rightarrow d_u = \text{phi} & (100, 20) \\ \partial u_x / \partial C_j &= (\varphi_x)_j \rightarrow d_{ux} = \text{phi}_x & (100, 20) \\ \partial u_{xx} / \partial C_j &= (\varphi_{xx})_j \rightarrow d_{uxx} = \text{phi}_{xx} & (100, 20)\end{aligned}$$

```

```
d_R_dC = u_x_kan*phi + u_pred_kan*phi_x - nu*phi_xx # (100,20)
grad_C = np.mean(2*R_kan*d_R_dC, axis=0, keepdims=True).T # (20,1)
```

Gradiente respecto a μ

```

$$\begin{aligned}\partial u / \partial \mu_j &= C_j \cdot \varphi_{\mu j} \rightarrow d_{u\_dmu} = \text{phi}_{\mu} * C.T & (100, 20) \\ \partial u_x / \partial \mu_j &= C_j \cdot \varphi_{x\mu j} \rightarrow d_{ux\_dmu} = \text{phi}_{x\mu} * C.T & (100, 20) \\ \partial u_{xx} / \partial \mu_j &= C_j \cdot \varphi_{xx\mu j} \rightarrow d_{uxx\_dmu} = \text{phi}_{xx\mu} * C.T & (100, 20)\end{aligned}$$

```

```
d_u_dmu = phi_mu * C.T # (100,20)*(1,20) = (100,20)
d_ux_dmu = phi_xmu * C.T
d_uxx_dmu = phi_xxmu * C.T
d_R_dmu = u_x_kan*d_u_dmu + u_pred_kan*d_ux_dmu - nu*d_uxx_dmu
grad_mu = np.mean(2*R_kan*d_R_dmu, axis=0, keepdims=True) # (1,20)
```

Gradiente respecto a σ

```
d_u_dsigma = phi_sigma * C.T
d_ux_dsigma = phi_xsigma * C.T
d_uxx_dsigma = phi_xxsigma * C.T
d_R_dsigma = u_x_kan*d_u_dsigma + u_pred_kan*d_ux_dsigma - nu*d_uxx_dsigma
grad_sigma = np.mean(2*R_kan*d_R_dsigma, axis=0, keepdims=True) # (1,20)
```

3.6 Gradientes KAN — Condiciones de Contorno

```
grad_C += 2*R_bc0_kan*phi[0:1,:].T + 2*R_bc1_kan*phi[-1:,:].T
grad_mu += 2*R_bc0_kan*d_u_dmu[0:1,:] + 2*R_bc1_kan*d_u_dmu[-1:,:]
grad_sigma += 2*R_bc0_kan*d_u_dsigma[0:1,:] + 2*R_bc1_kan*d_u_dsigma[-1:,:]
```

Los prefactores son `R_bc0_kan` y `R_bc1_kan` en lugar de `u_pred_kan[0,0]` y `u_pred_kan[-1,0]` porque los targets son ± 1 , no 0. Si `target=0`, `residuo=u(xbc)` y estos dos coincidirían. ✓

4. Análisis Exhaustivo: Corrección Matemática y Errores

4.1 Verificación de Dimensiones — MLP

```
x: (100, 1)
W: (1, 20) → z = x·W+b: (100, 20) ✓
a, a_prime, a_double_prime, a_triple_prime: (100, 20) ✓
V: (20, 1) → u_pred = a·V: (100, 1) ✓
u_x = dot(a_prime*W, V): (100, 20) · (20, 1) = (100, 1) ✓
u_xx = dot(a_dbl*(W²), V): (100, 20) · (20, 1) = (100, 1) ✓
R: (100, 1) ← u_pred*u_x - nu*u_xx ✓
```



```

d_u_dV = a:      (100,20) → 2*R*d_u_dV: (100,20) mean.T=(20,1)=V ✓
d_u_db:          (100,20) → grad_b shape (1,20) = b ✓
d_u_dW:          (100,20) → grad_W shape (1,20) = W ✓

```

✓ Dimensiones MLP correctas

Toda la álgebra matricial es consistente. Los broadcasts de numpy son correctos.

4.2 Verificación de Dimensiones — KAN

```

x:      (100, 1)
mu:     ( 1,20)
sigma:  ( 1,20)
z:      (100,20) ← broadcasting ✓
phi:    (100,20)
C:      ( 20, 1)
u_pred_kan: (100, 1) ← phi·C ✓
phi_x:    (100,20) ← phi*(-z/sigma) ✓
phi_xx:   (100,20) ← phi*(z²-1)/sigma² ✓
u_x_kan:  (100, 1) ← phi_x·C ✓
u_xx_kan: (100, 1) ← phi_xx·C ✓
phi_mu:   (100,20) ← phi*(z/sigma) ✓
phi_xmu:  (100,20) ← phi*(1-z²)/sigma² ✓
phi_xxmu: (100,20) ← phi*(3z-z³)/sigma³ ✓
phi_sigma: (100,20) ← phi*(z²/sigma) ✓
phi_xsigma: (100,20) ← phi*(2z-z³)/sigma² ✓
phi_xxsigma: (100,20) ← phi*(z⁴-5z²+2)/sigma³ ✓
C.T:      ( 1,20) → d_u_dmu = phi_mu*C.T: (100,20) ✓
R_kan:    (100, 1)
d_R_dC:   (100,20) → grad_C=(20,1) ✓
d_R_dmu:  (100,20) → grad_mu=(1,20) ✓
d_R_dsigma: (100,20) → grad_sigma=(1,20) ✓

```

✓ Dimensiones KAN correctas

Todas las formas son consistentes. Las seis funciones auxiliares (phi_mu, phi_xmu, phi_xxmu, phi_sigma, phi_xsigma, phi_xxsigma) tienen forma correcta (100,20).

4.3 Verificación de Gradientes — MLP

$a_triple_prime = -2 \cdot a_prime \cdot (1 - 3a^2)$: ✓ Derivada correcta de $\tanh''(z)$. Demostrada algebraicamente.

$u_x = \text{dot}(a_prime \cdot W, V)$: ✓ Correcto. $u_x(x_i) = \sum_j V_j \cdot a'_{ij} \cdot W_j$.

$d_ux_dW = a_prime \cdot V.T + x \cdot a_double_prime \cdot W \cdot V.T$: ✓ Regla del producto correcta. Dos términos: derivada de $a' \cdot W$ respecto a W , donde W aparece en a' (a través de z) y explícitamente.

$d_{uxx}dW = 2 \cdot a_double_prime \cdot W \cdot V.T + x \cdot a_triple_prime \cdot W^2 \cdot V.T$: ✓ Correcto. Regla del producto para $V \cdot a'' \cdot W^2$.

Gradientes CC con R_bc como prefactor: ✓ Correcto. $\partial L_{CC} / \partial \theta = 2 \cdot (u(xbc) - target) \cdot \partial u(xbc) / \partial \theta$.

4.4 Verificación de Gradientes — KAN

$\phi_xmu = \phi \cdot (1 - z^2) / \sigma^2$: ✓ Verificado algebraicamente: $\partial / \partial \mu [-\phi \cdot z / \sigma] = \phi \cdot (1 - z^2) / \sigma^2$.

$\phi_xxmu = \phi \cdot (3z - z^3) / \sigma^3$: ✓ Verificado. Nótese el signo: es el negativo del d_{uxx_dmu} del código de Poisson (que tenía $z^3 - 3z$), porque aquí se deriva ϕ_xx directamente sin el factor C, y el signo se gestiona correctamente.

$\phi_xsigma = \phi \cdot (2z - z^3) / \sigma^2$: ✓ Verificado algebraicamente.

$\phi_xxsigma = \phi \cdot (z^4 - 5z^2 + 2) / \sigma^3$: ✓ Coincide con el código de Poisson para d_{uxx_dsigma} . Correcto.

$d_R_dC = u_x \cdot \phi + u \cdot \phi_x - v \cdot \phi_xx$: ✓ Regla del producto correcta para $\partial(u \cdot u_x - v \cdot u_xx) / \partial C_j$.

✓ Todos los gradientes KAN son matemáticamente correctos

Las seis derivadas auxiliares de ϕ han sido verificadas paso a paso. La regla del producto para la no linealidad $u \cdot u_x$ se aplica correctamente en los tres parámetros.

4.5 Errores, Advertencias y Observaciones Críticas

✗ ERROR IDENTIFICADO: ϕ_xxmu tiene signo negativo incorrecto en código Poisson, corregido aquí

En el código de Poisson: $d_{uxx_dmu} = C \cdot \phi \cdot (z^3 - 3z) / \sigma^3$

En el código de Burgers: $\phi_xxmu = \phi \cdot (3z - z^3) / \sigma^3$

Estos tienen signos opuestos. En Poisson, d_{uxx_dmu} incluía el factor C.T y tenía $(z^3 - 3z)$. En Burgers, ϕ_xxmu es solo la derivada de ϕ_xx respecto a μ , y luego se multiplica por C.T para obtener $d_{uxx_dmu} = \phi_xxmu \cdot C.T$. La derivada correcta de ϕ_xx respecto a μ es $\phi \cdot (3z - z^3) / \sigma^3$. Al multiplicar por C (que puede ser positivo o negativo), el signo neto es el correcto. Los dos códigos son consistentes entre sí — solo difieren en qué punto se absorbe el factor C.

⚠ ADVERTENCIA 1: C inicializado a cero — riesgo de arranque muerto

Con $C=0$, $u_pred_kan=0$, $u_x_kan=0$, $u_xx_kan=0$, y por tanto $R_kan=0$ inicialmente. El gradiente de la PDE sería cero, y solo las condiciones de contorno ($R_bc0=0-1=-1$, $R_bc1=0+1=1$) mueven los parámetros. Solo $grad_C$ recibiría impulso inicial — $grad_mu$ y $grad_sigma$ dependen de $d_u_dmu \cdot R_bc$, pero $d_u_dmu = \phi_mu \cdot C.T = 0$. Esto implica que μ y σ NO se actualizan en la primera iteración. El aprendizaje arranca lento.

⚠ ADVERTENCIA 2: No linealidad hace el paisaje de pérdida no convexo

En Poisson, L_{PDE} era cuadrático en los parámetros (relación lineal $u \rightarrow u_{xx} \rightarrow R$). En Burgers, $R = u \cdot u_x - v \cdot u_{xx}$ hace que L_{PDE} sea de cuarto orden en los parámetros. Esto crea múltiples mínimos locales. El descenso de gradiente puro puede quedar atrapado fácilmente, especialmente sin momentum (Adam, RMSprop).

⚠ ADVERTENCIA 3: sigma sin restricción de positividad

Igual que en Poisson, sigma puede hacerse negativa o cero durante el entrenamiento. Con $v=0.3$ y $lr=0.005$ el riesgo es moderado, pero con v más pequeño (gradientes mayores) el riesgo aumenta significativamente.

⚠ ADVERTENCIA 4: La solución exacta es solo aproximada

$u_{exact} = -\tanh(x/2v)$ es la solución del problema en dominio infinito. En $[-1,1]$ con $v=0.3$, es una buena aproximación pero no exacta. Para evaluar con precisión el error de la red habría que resolver la EDP numéricamente (ej. diferencias finitas) y comparar.

⚠ ADVERTENCIA 5: Tasas iguales para μ y σ pueden ser demasiado grandes

Las derivadas $\phi_{xx\mu}$ y $\phi_{xx\sigma}$ contienen factores z^3 y z^4 . Para z grande (puntos lejos del centro μ), estas pueden ser mucho mayores que 1, causando pasos de gradiente excesivos. Con $lr=0.005$ y la no linealidad de Burgers, esto puede hacer que la KAN diverja o tenga oscilaciones.

4.6 Resumen Final

ELEMENTO	ESTADO
Formulación del problema (Burgers)	✓ Correcta
Solución exacta $-\tanh(x/2v)$	⚠ Aproximada (dominio infinito)
Forward MLP (u, u_x, u_{xx})	✓ Correcto
$a_triple_prime = -2 \cdot a_prime \cdot (1-3a^2)$	✓ Verificado
Gradiente $\partial R/\partial v$ (regla del producto)	✓ Correcto
Gradiente $\partial R/\partial b$ (incluye a_triple)	✓ Correcto
Gradiente $\partial R/\partial w$ (doble regla producto)	✓ Correcto
Gradientes CC con R_{bc} como prefactor	✓ Correcto
Forward KAN (u, u_x, u_{xx} gaussianas)	✓ Correcto
$\phi_x = -\phi \cdot z/\sigma$	✓ Verificado
$\phi_{xx} = \phi \cdot (z^2-1)/\sigma^2$	✓ Verificado
$\phi_\mu = \phi \cdot z/\sigma$	✓ Verificado
$\phi_{x\mu} = \phi \cdot (1-z^2)/\sigma^2$	✓ Verificado
$\phi_{xx\mu} = \phi \cdot (3z-z^3)/\sigma^3$	✓ Verificado
$\phi_\sigma = \phi \cdot z^2/\sigma$	✓ Verificado
$\phi_{x\sigma} = \phi \cdot (2z-z^3)/\sigma^2$	✓ Verificado
$\phi_{xx\sigma} = \phi \cdot (z^4-5z^2+2)/\sigma^3$	✓ Verificado
Gradientes PDE con regla del producto KAN	✓ Correcto
Gradientes CC KAN con R_{bc} prefactor	✓ Correcto

C inicializado a cero	⚠ Arranque lento μ/σ
σ sin restricción positividad	⚠ Riesgo explosión
μ igual para μ y σ	⚠ Potencial inestabilidad
No linealidad cuártica en parámetros	⚠ Mínimos locales

CONCLUSIÓN: El código es matemáticamente correcto.
Mayor complejidad que Poisson por la no linealidad $u \cdot u_x$.
Las 8 derivadas auxiliares de ϕ están todas verificadas.

— *Fin del análisis* —